Final Report

ENAE450 - Robotics Programming

Student Name: Dror Kosowsky-Sachs, Drew Wei-Williams, Ethan Stein
Date: May 15, 2026

1 Introduction

1.1 The Challenge

The assignment was to prepare the robot to escape a maze in a limited amount of time. There are two races. In the first, the robot is allowed to map the entire maze and then must autonomously navigate its way out. In the second race, the robot has no map and needs to explore the maze on its own and find the exit. In each race, the robot must execute a 180 degree turn after reaching the exit.

1.2 Drive & Code Files

- ROS2 Bag Files: https://drive.google.com/drive/folders/1-4maujw3YoTPvjLRRXFxZ_AvLviO1IyL?usp=drive_link
- Video Files: https://drive.google.com/drive/folders/1bKdKFVnOniPNTlQOssgtiU-kqpCHbhgL?usp=drive_link
- Code: code.umd.edu/drorks/enae450_s26

2 Hardware: Methods, Results

2.1 Part 1: Nav2

We used the standard ROS2 packages with TurtleBot 4 during this part. The process was: SLAM, creating and saving a map, localization, and autonomous navigation. SLAM uses LiDAR to take scans of the environment using laser pulses and to create a map. We used teleoperation to move the robot around and had SLAM simultaneously expand the map by reading the robot's odometry and detect its surroundings, and continued until the entire maze was mapped.

Once the map was complete, we used Adaptive Monte Carlo Localization (AMCL) with a pose estimate that we placed on RViz in order to place the robot on the already existing map. This uses a particle filter, which goes through guesses, or particles, that find the robot's position. Nav2 then created a costmap of all of the obstacles in the map, which not only marks any obstacles, but also inflates them by the robot's radius so as to avoid collisions with the perimeter of the robot. Finally, after setting a Nav2 goal on RViz, the robot was able to autonomously navigate through the maze.

2.2 Part 2: Autonomous Maze Solver

In our script, we used LiDAR to sense the surrounding objects and the cmd_vel topic in order to publish linear and angular velocities so that the robot can navigate the maze. We had many parameters that needed to be fine tuned in order to detect and move in the most efficient and effective way possible, including setting a desired distance from the right hand wall and parameters for how closely to control that distance.

We used two main functions to control the robot. One was scan_callback which updates every time LiDAR scans. This is where most of the logic is being done. The other function, timer_callback, is called 20 times a second. This is used mainly for publishing and moving the robot.

The robot uses a geometric heading algorithm to detect the angle its heading is making with the wall, which usually should be parallel. The following is a visual that explains the math in calculating a geometric heading error that we made using ChatGPT:

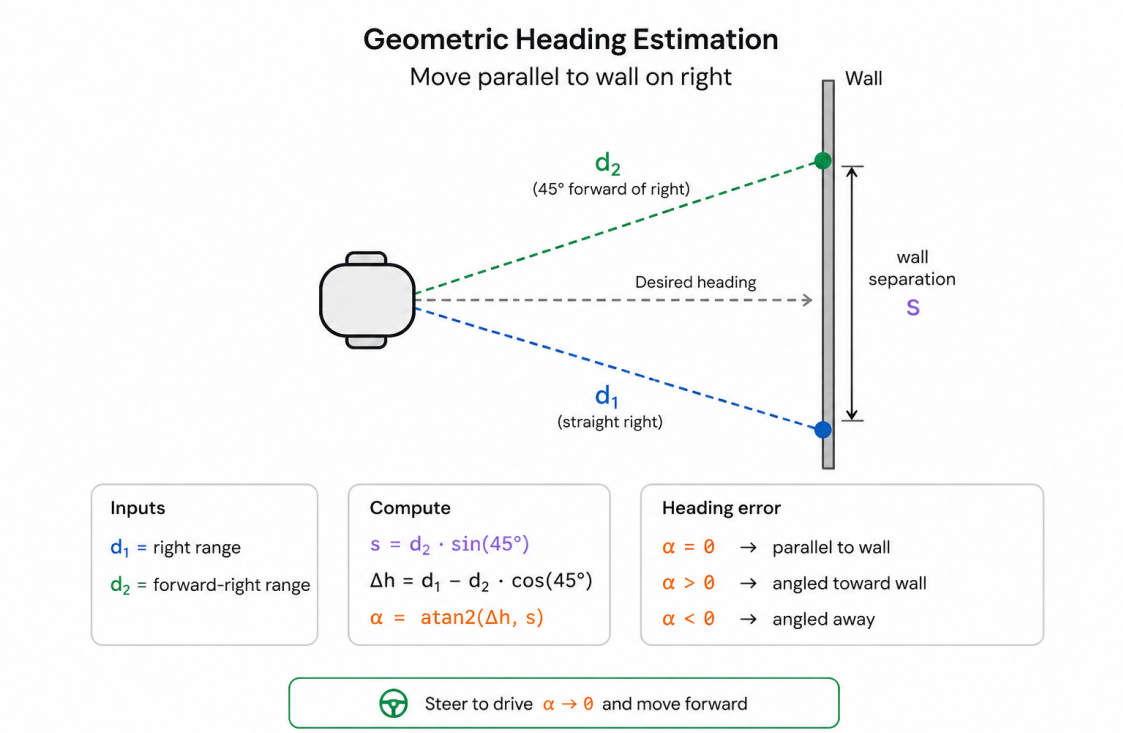


Figure 1: Visual of Geometric Heading Estimation (Source: ChatGPT)

This is a snippet of the code which produces an "angle error", and shows how we implemented the math from the visual in our code:

```
offset_rad = math.radians(45)
separation = front_right * math.sin(offset_rad)
height_diff = right - front_right * math.cos(offset_rad)
angle_error = math.atan2(height_diff, separation)
```

The computation allows the control to have a clear understanding of its heading angle with the wall.

It was originally assumed that the forward facing angle was 0 radians, but we discovered that the LiDAR is rotated, making forward $-\pi/2$ and right $-\pi$.

There are 3 different states during the maze solving that the robot goes through in order to complete the assignment. It starts in WALL_FOLLOWING and when it finishes the maze it goes into U_TURN, and then once the u-turn is complete it goes into the FINISHED state.

The WALL_FOLLOWING state is by far the most complex. In that state, we classified the robot into many different cases to figure out what it should be looking for and how it should be moving. The most simple case is case 3. If there's a wall detected on the right, follow it. Case 2 is if there's no wall detected on the right. In this case, we have the robot move towards the right until it finds something. Case 1 is when it's following the right wall but comes to an obstacle in front and has to turn left. We also added a "most pressing case", which comes first in the code, where the robot is about to hit an object, and must execute a rotation to avoid imminent collision. In addition, as part of the check for case 1, the LiDAR looked to the front left, which is where a left turn would take the robot, to see if a left turn would cause a collision.

The ordering of the cases is in numerical order, meaning it first checks if the space directly ahead is clear, then whether a wall on the right exists, and then goes ahead only if both are true.

For exit detection, the robot checks whether there are objects in front of it, and if none are present will initiate a U-turn. In our code, the robot is often turning right as it leaves the maze to continue following the wall, resulting in some of the wall appearing in the front, 180 degree scan and preventing exit initiation. As a result, we only check the area 23 degrees forward of left until 60 degrees forward of right, meaning an approximately 97 degree window. This proved to be successful in our testing. After successfully detecting an exit, the U-turn is done by turning its heading for about a second.

The code on the github has been commented to show how each of these was implemented.

3 ros2_bag Plots

Note that the ros2_bag files can be found here: https://drive.google.com/drive/folders/1-4maujw3YoTPvjLRRXFxZ_AvLviO1IyL?usp=drive_link

3.1 Plot: Part 1 REPRESENTATION

Note that this plot is representative of the functionality of the TurtleBot4 and is not the actual map tested on.

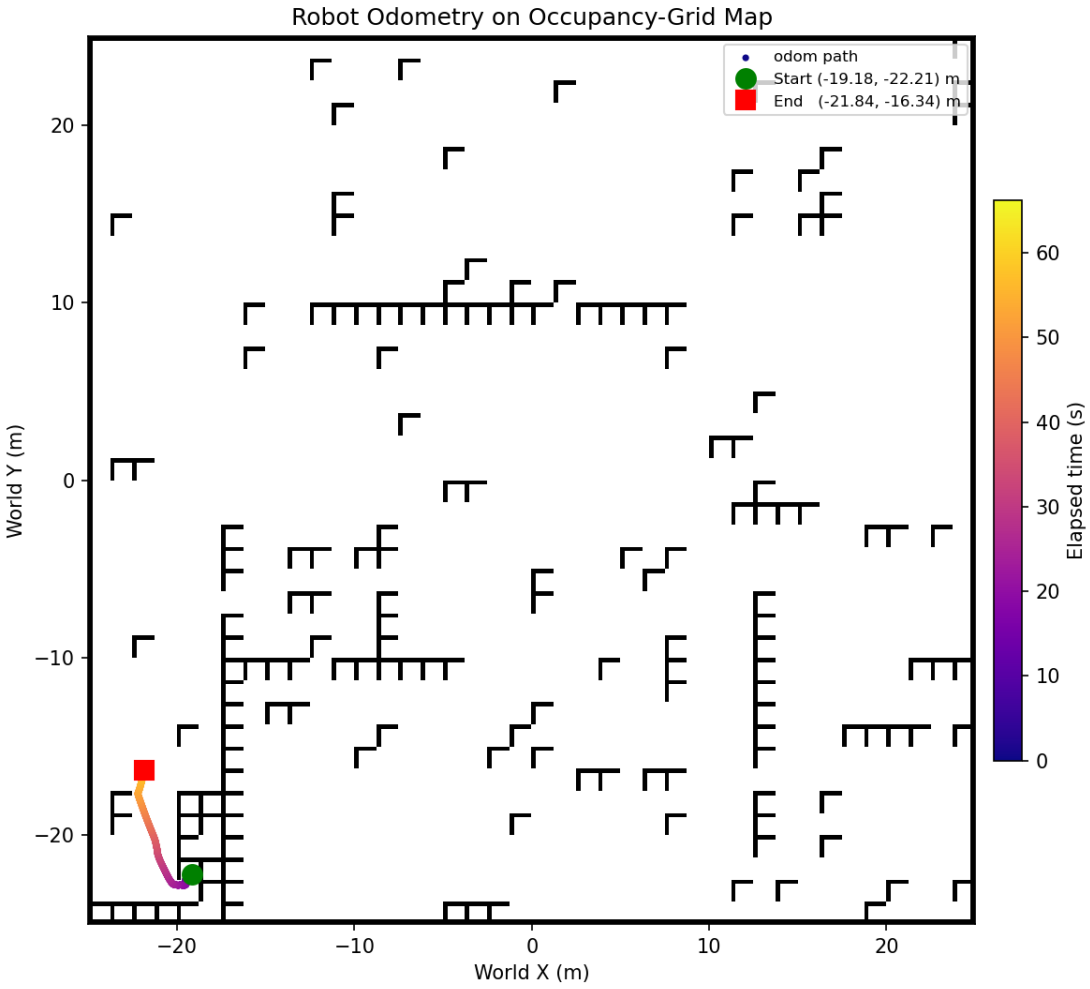


Figure 2: 2D TurtleBot4 simulation running goal seeking

This graph simulates what an obstacle avoidance demonstrated by Nav2 goal seeking would look like. As is seen in the figure, the robot receives a goal for a certain location, and creates an optimized route toward the goal, avoiding obstacles along the way. After reaching the goal, the robot initiates the ending sequence, meaning completing a 180 degree turn and printing "finished" on the logger.

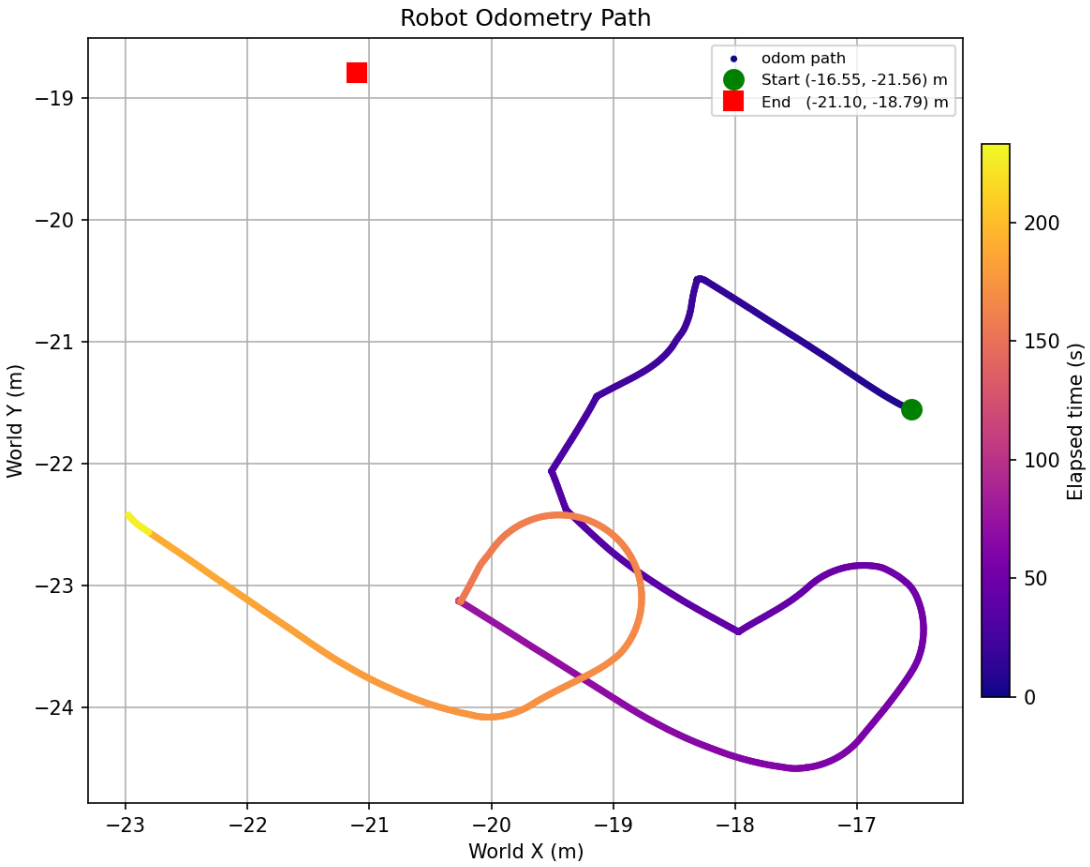


Figure 3: Unmapped race, two overlayed attempts

This graph shows two attempts we made for the second, unmapped race. In the first attempt (as shown in the time coloration from blue to purple), the robot navigated the maze and then began following the outside wall due to issues in exit recognition. In the second attempt (from orange to yellow) the robot started at the exit of the maze, and then followed the outside wall until the exit recognition code activated.

4 Retrospective

4.1 What worked

Our right hand wall follower worked quite well. The robot was able to follow the right hand easily early on.

Originally, the robot didn't look exactly to the right, but rather slightly ahead to the right so as to predict turns before they must be made, but this resulted in hitting the wall on right turns,

so it was removed.

The robot was able to do left turns successfully early on, though we ran into issues at first in an "alleyway" situation, where the robot must essentially turn around to avoid a wall. We fixed this by adding a check that looks to the front left, and ensures that the area it will turn left into is also clear. The robot was better equipped to solve this type of obstacle afterward.

4.2 What didn't Work

We made a script for the robot to find the most open space and go towards it until it got out of the maze. The problem with this solution was that it could not tell where it had been and where it was going consistently, so it would go in circles, thinking that going backwards could be the way out of the maze.

In addition since we did not make a launch file for the mapped race, we found it very tedious and time consuming to open SLAM, RViz, save a map, localization, and then open Nav2 any time we wanted to set a goal. This could likely be done much faster with a launch file, and also improve the reliability of completing each step correctly and in the proper order.

Our end sequence for the unmapped run also only worked some of the time, as it was hard to differentiate between readings of maze walls and random objects that were somewhat near the maze opening.

We tried to add a "cleared exit counter", which would count the number of times in a row that the robot detects empty space in front of it, and only commence the U-turn once a threshold has been passed. In theory, this would limit false positives and only start a U-turn when it actually exits the maze. However, we found that the robot rarely had false positives for exit detection within the maze, so this feature was not necessary.

4.3 What can be changed

We would make a launch file that executes each step of part 1 successively to save time.

We would also try to improve the goal seeking logic for part two by logically removing where problematic goals would be, such as "back the way you came", and would probably be able to finish the maze far quicker.

For the end sequence, we would also implement a piece of logic that differentiates between objects that continuously appear over a longer window in a predictable way and objects that appear in a small one. This could allow one to distinguish between a wall which predictably appears over a longer window and in a predictable way, and say, a chair leg from a table that happens to be visible from the maze exit. This could improve reliability in recognizing maze openings.

5 Team contribution summary describing who did what

We all worked on the code and troubleshot it together during lab hours. Specifically, we used each person's computer at times when one seemed to have more consistency with the robot than another. In addition, we all contributed to writing the report and assembling the submission materials. Dror and Drew were responsible for the retrospective, and Dror and Ethan contributed more to the Hardware section.

References

- ROS2 Documentation: https://docs.ros.org

- NumPy Documentation: https://numpy.org/doc/stable/
- Professor's GitLab: https://code.umd.edu/conroyj/enae450_s26
- RosBag Documentation: https://wiki.ros.org/rosbag
- TurtleBot4 Documentation: https://turtlebot.github.io/turtlebot4-user-manual/
- Adaptive Monte Carlo Localization (AMCL): https://en.wikipedia.org/wiki/Monte_Carlo_localization
- Dror's Github (with code and documents): https://code.umd.edu/drorks/enae450_s26